

Multi-Agent Analytics Assistant - System Architecture

Table of Contents

- [Overview](#)
 - [Key Features](#)
 - [High-Level Architecture](#)
 - [System Flow Diagram](#)
 - [Component Details](#)
 - [Agent Workflow](#)
 - [Data Flow](#)
 - [System Capabilities](#)
 - [Technical Implementation Details](#)
 - [Technology Stack](#)
 - [Recent Enhancements](#)
 - [System Metrics](#)
 - [Extensibility & Roadmap](#)
 - [Deployment Guide](#)
-

Key Features

Intelligent Question Routing

- Automatic classification of descriptive (WHAT) vs causal (WHY) questions
- Context-aware agent selection and orchestration
- Support for complex multi-step analytical workflows

Multi-Agent Architecture

1. **Planner Agent:** Routes questions intelligently
2. **Text-to-SQL Agent:** Converts natural language to PostgreSQL
3. **Visualization Agent:** Generates interactive Plotly charts
4. **Hypothesis Agent:** Creates testable hypotheses for causal questions
5. **Statistical Agent:** Performs rigorous statistical testing

Advanced Analytics

- **Descriptive Analytics:** Summaries, distributions, comparisons
- **Causal Analytics:** Hypothesis testing with statistical validation
- **Automated Insights:** AI-generated business recommendations
- **Multi-Modal Output:** SQL, data, visualizations, statistical results

Multi-Dataset Support

- Seamlessly switch between HR and Sales datasets

- Dynamic schema loading and context adaptation
- Extensible architecture for new domains
- Dataset-specific example prompts and documentation

Statistical Rigor

- Chi-Square Test for categorical associations
- Independent t-Test for two-group comparisons
- One-Way ANOVA for multi-group analysis
- Pearson/Spearman correlation for numerical relationships
- Effect size calculations and confidence intervals

Smart Visualizations

- Automatic chart type selection based on data shape
- Intelligent number formatting (currency, percentages, counts)
- Ordinal scale detection for rating variables
- Interactive Plotly charts with zoom, pan, and export

Real-Time Monitoring

- Enhanced agent activity tracking
- Performance metrics and timing data
- Color-coded log levels (info, success, warning, error)
- Expandable error details with stack traces

AI-Powered Insights

- LLM-generated interpretations of statistical results
- Plain English explanations of technical findings
- Actionable business recommendations
- Limitation awareness and caveats

Enterprise-Ready

- Secure Azure OpenAI integration
- SQL injection prevention
- Input validation with Pydantic
- CORS configuration for security
- Comprehensive error handling

Overview

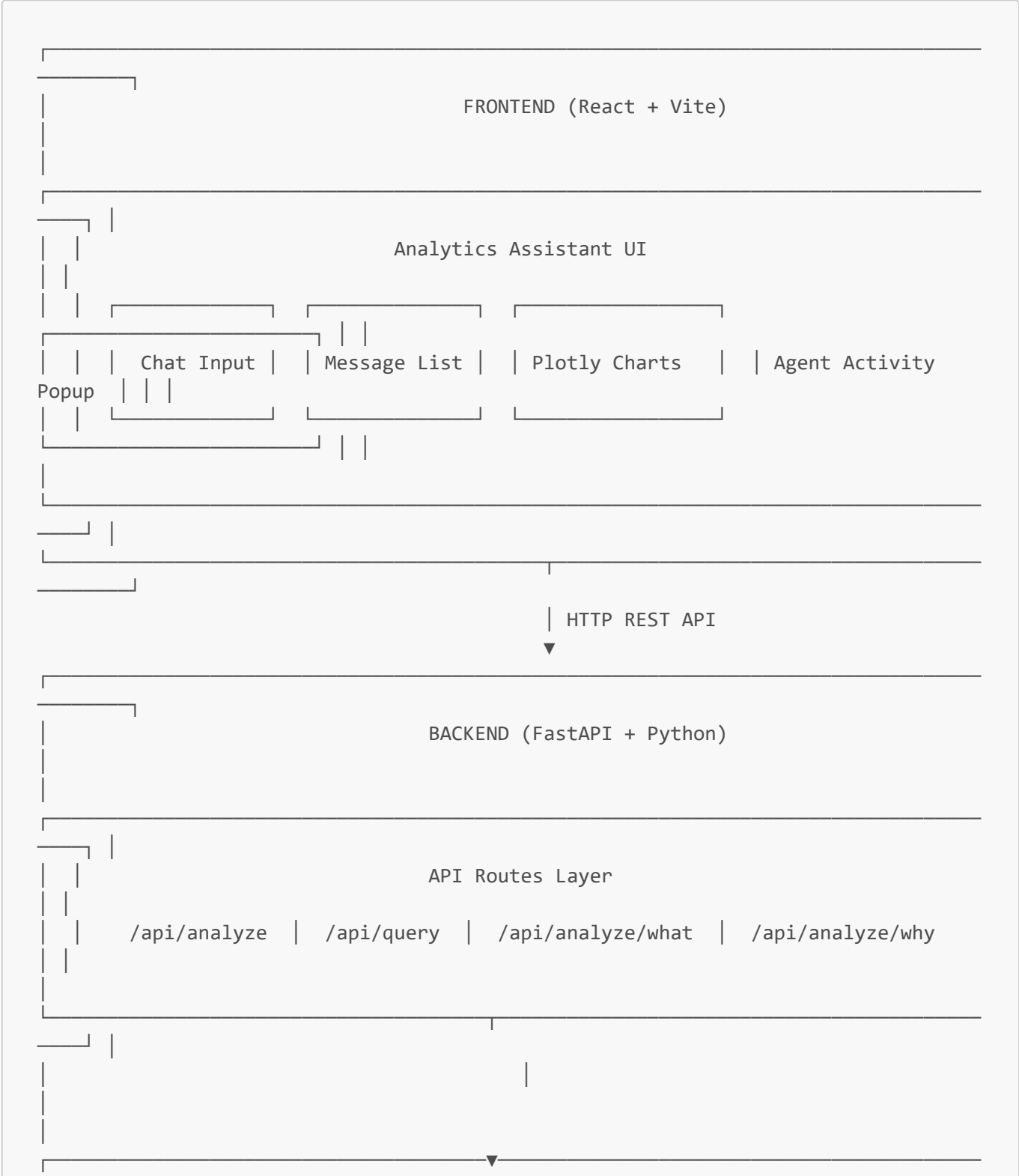
The **Multi-Agent Analytics Assistant** is an intelligent analytics system that uses multiple specialized AI agents to answer both **descriptive (WHAT)** and **causal (WHY)** questions about business data. The system features:

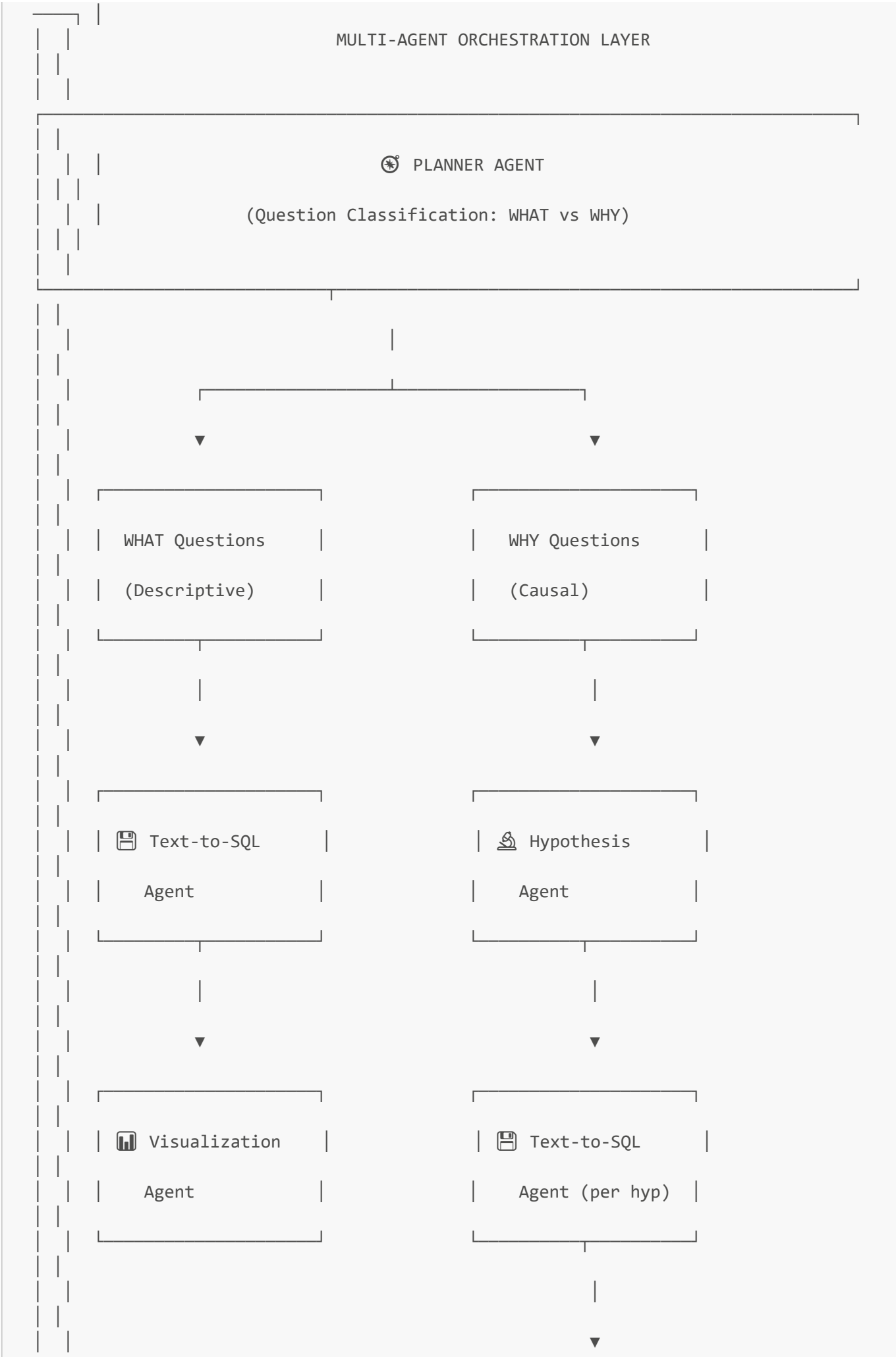
-  **Multi-Agent Architecture:** 5 specialized AI agents working in orchestration
-  **Dual Analytics:** Descriptive (WHAT) and Causal (WHY) analysis capabilities

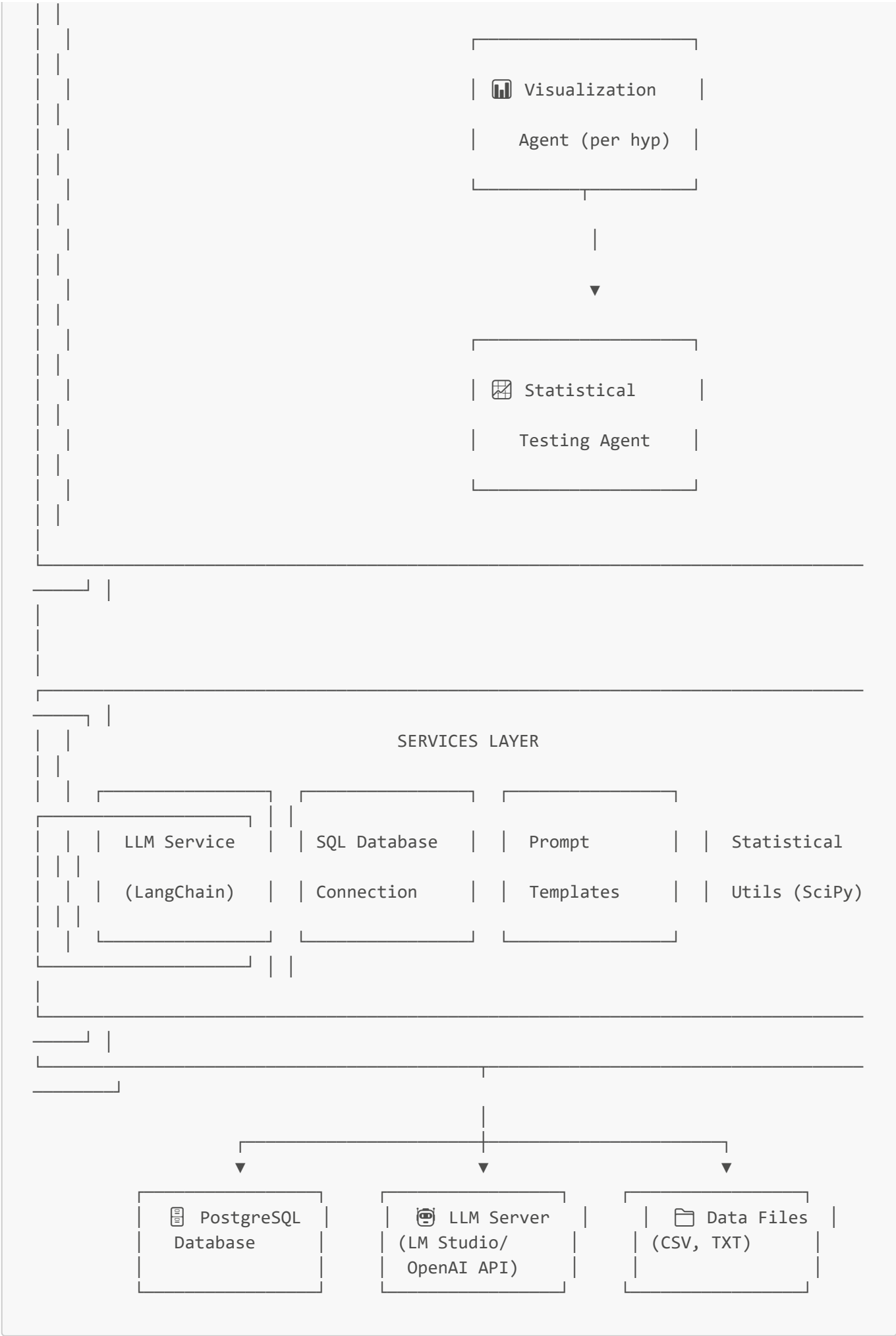
-  **Multi-Dataset Support:** Seamless switching between HR and Sales datasets
-  **Statistical Rigor:** Automated hypothesis testing with Chi-square, t-test, ANOVA, and correlation analysis
-  **Smart Visualizations:** LLM-generated Plotly charts with automatic type selection
-  **Real-time Monitoring:** Enhanced agent activity tracking with performance metrics
-  **AI-Powered Insights:** LLM-generated business interpretations of statistical results

The system automatically routes questions to appropriate agents based on intelligent question classification.

High-Level Architecture







System Flow Diagram

```

flowchart TB
    subgraph Frontend["🖥️ FRONTEND (React + Vite + TypeScript)"]
        UI[AnalyticsAssistant.tsx
Main Page Component]
        ChatInput[ChatInput.jsx
User Input]
        MessageList[MessageList.jsx
Chat Display]
        Plot[react-plotly.js
Visualizations]
        AgentPopup[AgentActivityPopup.tsx
Agent Status]
        APIClient[client.js
HTTP Client]
    end

    subgraph Backend["⚙️ BACKEND (FastAPI + Python)"]
        subgraph API["🌐 API Layer"]
            Routes[routes.py
FastAPI Router]
            AnalyzeEndpoint[/api/analyze]
            QueryEndpoint[/api/query]
            WhatEndpoint[/api/analyze/what]
            WhyEndpoint[/api/analyze/why]
        end

        subgraph MAS["👤 Multi-Agent System"]
            Orchestrator[multi_agent_system.py
Agent Orchestrator]

            subgraph Agents["Specialized Agents"]
                Planner[🧠 Planner Agent
Question Router]
                TextToSQL[📄 Text-to-SQL Agent
NL to SQL]
                Viz[📊 Visualization Agent
Plotly Generator]
                Hypothesis[🔍 Hypothesis Agent
Hypothesis Generator]
                Stats[📈 Stats Agent
Statistical Testing]
            end
        end

        subgraph Services["🔗 Services Layer"]
            LLM[llm.py
LLM Client]
            Config[config.py
Settings]
        end
    end

```

```

    end

    subgraph Utils["🔧 Utilities"]
        SQLUtils[text_to_sql_utils.py
DB & SQL Utils]
        VizUtils[visualization_utils.py
Chart Helpers]
        HypUtils[hypothesis_utils.py
JSON Parsing]
        StatsUtils[stats_utils.py
Statistical Tests]
    end

    subgraph Prompts["📝 Prompt Templates"]
        PlannerPrompt[Planner Prompt
WHAT vs WHY Classification]
        SQLPrompt[Text-to-SQL Prompt
PostgreSQL Generation]
        VizPrompt[Visualization Prompt
Plotly Code Gen]
        HypPrompt[Hypothesis Prompt
Bivariate Hypotheses]
        StatsPrompt[Stats Prompt
Test Selection]
    end

    end

    subgraph External["🌐 External Services"]
        DB[(🗄️ PostgreSQL
HR Data)]
        LLMServer[🖥️ LLM Server
LM Studio / OpenAI]
        DataFiles[📁 Data Files
CSV + TXT]
    end

    %% Frontend connections
    UI --> ChatInput
    UI --> MessageList
    UI --> Plot
    UI --> AgentPopup
    ChatInput --> APIClient
    APIClient --> Routes

    %% API to MAS
    Routes --> AnalyzeEndpoint
    Routes --> QueryEndpoint
    Routes --> WhatEndpoint
    Routes --> WhyEndpoint
    AnalyzeEndpoint --> Orchestrator
    QueryEndpoint --> TextToSQL
    WhatEndpoint --> TextToSQL
    WhyEndpoint --> Hypothesis

```

```

%% MAS Flow
Orchestrator --> Planner
Planner -->|WHAT| TextToSQL
Planner -->|WHY| Hypothesis
TextToSQL --> Viz
Hypothesis --> TextToSQL
Hypothesis --> Stats

%% Services
Agents --> LLM
Agents --> Prompts
LLM --> LLMServer

%% Utils
TextToSQL --> SQLUtils
Viz --> VizUtils
Hypothesis --> HypUtils
Stats --> StatsUtils
SQLUtils --> DB
StatsUtils --> DB
HypUtils --> DataFiles

```

Component Details

Frontend Components

```

frontend-repo/src/
├── App.jsx                # Root application component
├── main.jsx              # React entry point
├── styles.css            # Global styles (Tailwind CSS)
├── api/
│   ├── client.js        # API client functions
│   │   ├── sendAnalyticsQuery() # Main analytics endpoint
│   │   ├── sendChat()      # Chat endpoint
│   │   └── sendDirectSQLQuery() # Direct SQL endpoint
├── components/
│   ├── AgentActivity.tsx # Agent status badges
│   ├── AgentActivityPopup.tsx # Real-time agent logs modal
│   ├── ChatInput.jsx     # User input component
│   ├── CodeBlock.tsx    # SQL/Python code display
│   ├── MessageList.jsx  # Chat message list
│   └── Tabs.tsx         # Tab navigation component
├── lib/
│   └── api.ts           # TypeScript types & constants
└── pages/
    └── AnalyticsAssistant.tsx # Main analytics page (928 lines)

```


- └─ State Management # Messages, history, loading states
- └─ handleSubmit() # Query submission handler
- └─ Response Rendering # WHAT/WHY specific rendering
- └─ Plotly Integration # Chart rendering

Backend Components

```

backend-repo/app/
├─ main.py # FastAPI application entry
│   └─ CORS Configuration
│   └─ Startup/Shutdown events
│   └─ Multi-Agent initialization
├─ config.py # Application settings
│   └─ Database config
│   └─ LLM config
│   └─ CORS origins
├─ api/
│   └─ routes.py # API endpoints (480 lines)
│       └─ POST /api/chat # General chat endpoint
│       └─ POST /api/query # Direct SQL query
│       └─ POST /api/sql-only # SQL without visualization
│       └─ POST /api/analyze # Main multi-agent endpoint
│       └─ POST /api/analyze/what # Direct WHAT questions
│       └─ POST /api/analyze/why # Direct WHY questions
├─ services/
│   └─ llm.py # LLM client (Azure OpenAI)
│   └─ dataset_manager.py # Multi-dataset management (NEW)
│   └─ multi_agent_system.py # Agent orchestration (386 lines)
│       └─ process_question() # Main entry point
│       └─ _handle_what_question()
│       └─ _handle_why_question()
│       └─ initialize_llm()
│   └─ planner_agent.py # Question classification
│   └─ text_to_sql_agent.py # NL to SQL conversion (209 lines)
│   └─ visualization_agent.py # Plotly code generation (153 lines)
│   └─ hypothesis_agent.py # Hypothesis generation (204 lines)
│   └─ stats_agent.py # Statistical testing + LLM interpretation (222
lines)
├─ prompts/
│   └─ prompts.py # All prompt templates (344 lines)
│       └─ planner_agent_prompt
│       └─ text_to_sql_agent_prompt
│       └─ visualization_agent_prompt
│       └─ hypothesis_agent_prompt
│       └─ stats_agent_prompt
└─ utils/

```

```

├─ logger.py                # Enhanced logging system (NEW)
├─ logging_integration_example.py # Integration examples (NEW)
├─ text_to_sql_utils.py      # DB connection, SQL utilities
├─ visualization_utils.py   # DataFrame processing, code extraction
├─ hypothesis_utils.py      # JSON response parsing
├─ stats_utils.py           # Statistical test functions (464 lines)
│   ├── chi_square_test()    # Categorical vs Categorical
│   ├── t_test()             # Categorical (2) vs Numerical
│   ├── anova_test()         # Categorical (3+) vs Numerical
│   ├── pearson_correlation()# Numerical vs Numerical
│   └── spearman_correlation()

```

Agent Workflow

WHAT Questions (Descriptive Analytics)

sequenceDiagram

```

participant U as User
participant FE as Frontend
participant API as FastAPI
participant P as Planner Agent
participant SQL as Text-to-SQL Agent
participant V as Visualization Agent
participant DB as PostgreSQL
participant LLM as LLM Server

```

U->>FE: "What is the attrition rate by department?"

FE->>API: POST /api/analyze

API->>P: Classify question

P->>LLM: Analyze question type

LLM-->>P: {"question_type": "WHAT"}

P-->>API: Route to Text-to-SQL + Viz

API->>SQL: Generate SQL

SQL->>LLM: Convert NL to SQL

LLM-->>SQL: SELECT ... FROM hr_data

SQL->>DB: Execute query

DB-->>SQL: Result DataFrame

SQL-->>API: {sql, data, rows}

API->>V: Generate visualization

V->>LLM: Generate Plotly code

LLM-->>V: Python code

V->>V: Execute code

V-->>API: {figure, code}

API-->>FE: {sql, data, visualization}

FE->>FE: Render chart + data table

FE-->>U: Display results

WHY Questions (Causal Analytics)

sequenceDiagram

```

    participant U as User
    participant FE as Frontend
    participant API as FastAPI
    participant P as Planner Agent
    participant H as Hypothesis Agent
    participant SQL as Text-to-SQL Agent
    participant V as Visualization Agent
    participant S as Stats Agent
    participant DB as PostgreSQL
    participant LLM as LLM Server
  
```

```

    U->>FE: "Why do employees leave?"
    FE->>API: POST /api/analyze
    API->>P: Classify question
    P->>LLM: Analyze question type
    LLM-->>P: {"question_type": "WHY"}
    P-->>API: Route to Hypothesis pipeline
  
```

Note over API,H: Step 1: Generate Hypotheses

```

    API->>H: Generate hypotheses
    H->>LLM: Generate 3 testable hypotheses
    LLM-->>H: Hypotheses JSON
    H-->>API: [{H1}, {H2}, {H3}]
  
```

Note over API,V: Step 2: Get Data & Visualize (per hypothesis)

```

    loop For each hypothesis
      API->>SQL: Generate SQL for hypothesis
      SQL->>LLM: Convert to SQL
      LLM-->>SQL: SELECT query
      SQL->>DB: Execute
      DB-->>SQL: DataFrame

      API->>V: Visualize hypothesis data
      V->>LLM: Generate Plotly code
      LLM-->>V: Python code
      V-->>API: Figure
    end
  
```

Note over API,S: Step 3: Statistical Testing

```

    API->>S: Run statistical tests
    S->>DB: Load full dataset
    S->>S: Execute appropriate tests
    Note right of S: Chi-square, t-test,
    ANOVA, Correlation
    S-->>API: Test results + interpretations
  
```

```

    API-->>FE: {hypotheses, visualizations, stats}
    FE->>FE: Render all results
    FE-->>U: Display analysis
  
```

Data Flow

Request/Response Structure

```
flowchart LR
    subgraph Request["📄 Request"]
        Q[Question String]
        Opts[Options]
        num_hypotheses
        include_visualization
    end

    subgraph Processing["⚙️ Processing"]
        Class[Classification]
        WHAT / WHY
        Agent[Agent Routing]
        Exec[Execution]
    end

    subgraph Response["📄 Response"]
        subgraph WHAT_Resp["WHAT Response"]
            SQL1[SQL Query]
            Data1[Data Array]
            Viz1[Plotly JSON]
        end

        subgraph WHY_Resp["WHY Response"]
            Hyp[Hypotheses Array]
            SQLs[SQL Queries Array]
            Vizs[Visualizations Array]
            Stats[Statistical Results]
            Sum[Summary]
        end
    end

    Q --> Class
    Opts --> Agent
    Class --> Agent
    Agent --> Exec
    Exec -->|WHAT| WHAT_Resp
    Exec -->|WHY| WHY_Resp
```

Database Schemas (Multi-Dataset Support)

Schema 1: HR Employee Attrition (hr_data.employee_attrition)

--

hr_data.employee_attrition			
Column Name	Type	Category	Description
age	INTEGER	Numerical	Employee age
attrition	TEXT	Categorical	Yes/No left company
businesstravel	TEXT	Categorical	Travel frequency
dailyrate	INTEGER	Numerical	Daily salary rate
department	TEXT	Categorical	Work department
distancefromhome	INTEGER	Numerical	Commute distance
education	INTEGER	Ordinal	Education level 1-5
educationfield	TEXT	Categorical	Field of study
environmentsatisfaction	INTEGER	Ordinal	Satisfaction 1-4
gender	TEXT	Categorical	Male/Female
hourlyrate	INTEGER	Numerical	Hourly wage
jobinvolvement	INTEGER	Ordinal	Involvement 1-4
joblevel	INTEGER	Ordinal	Position level 1-5
jobrole	TEXT	Categorical	Job title
jobsatisfaction	INTEGER	Ordinal	Satisfaction 1-4
maritalstatus	TEXT	Categorical	Marital status
monthlyincome	INTEGER	Numerical	Monthly salary
monthlyrate	INTEGER	Numerical	Monthly billing
numcompaniesworked	INTEGER	Numerical	Previous companies
overtime	TEXT	Categorical	Works overtime Y/N
percentsalaryhike	INTEGER	Numerical	Salary increase %
performancerating	INTEGER	Ordinal	Rating 1-4
relationshipsatisfaction	INTEGER	Ordinal	Satisfaction 1-4
stockoptionlevel	INTEGER	Ordinal	Stock options 0-3
totalworkingyears	INTEGER	Numerical	Total experience
trainingtimeslastyear	INTEGER	Numerical	Training sessions
worklifebalance	INTEGER	Ordinal	Balance rating 1-4
yearsatcompany	INTEGER	Numerical	Company tenure
yearsincurrentrole	INTEGER	Numerical	Role tenure
yearssincelastpromotion	INTEGER	Numerical	Time since promo
yearswithcurrmanager	INTEGER	Numerical	Manager tenure

Records: 1,470 employees | **Attrition Rate:** ~16%

Schema 2: Zalando Sales (sales_data.zalando_sales)

sales_data.zalando_sales			
Column Name	Type	Category	Description
order_id	TEXT	Identifier	Unique order ID
order_date	DATE	Temporal	Order timestamp
customer_id	TEXT	Identifier	Customer ID

	customer_name	TEXT	Categorical	Customer name
	customer_age	INTEGER	Numerical	Customer age
	gender	TEXT	Categorical	Male/Female
	city	TEXT	Categorical	Order city
	customer_segment	TEXT	Categorical	New/Returning/VIP
	product_id	TEXT	Identifier	Product SKU
	product_name	TEXT	Categorical	Product title
	category	TEXT	Categorical	Product category
	sub_category	TEXT	Categorical	Sub-category
	brand	TEXT	Categorical	Brand name
	price	NUMERIC	Numerical	Unit price
	quantity	INTEGER	Numerical	Units ordered
	discount_percent	NUMERIC	Numerical	Discount %
	revenue	NUMERIC	Numerical	Total revenue
	cost	NUMERIC	Numerical	Product cost
	profit	NUMERIC	Numerical	Order profit
	shipping_cost	NUMERIC	Numerical	Shipping cost
	order_status	TEXT	Categorical	Delivered/Returned
	delivery_time_days	INTEGER	Numerical	Delivery duration
	product_rating	NUMERIC	Numerical	Customer rating

Records: 10,000+ transactions | **Time Period:** 2023-2024

Technology Stack

Frontend

Technology	Purpose	Version
React	UI Framework	18.2.0
TypeScript	Type Safety	5.3.3
Vite	Build Tool	5.0.8
Tailwind CSS	Styling	3.3.5
Plotly.js	Charts	3.1.2
react-plotly.js	React Plotly	2.6.0
Headless UI	Accessible Components	1.7.17
Heroicons	Icons	2.1.1

Backend

Technology	Purpose	Version
FastAPI	Web Framework	Latest
Python	Language	3.12+

Technology	Purpose	Version
LangChain	LLM Orchestration	Latest
OpenAI SDK	LLM Client	Latest
Pandas	Data Processing	Latest
Plotly	Visualization	Latest
SciPy	Statistical Testing	Latest
psycopg2	PostgreSQL Driver	Latest
SQLAlchemy	ORM	Latest
Pydantic	Data Validation	Latest

Infrastructure

Component	Technology
Database	PostgreSQL (Multi-schema support)
LLM Provider	Azure OpenAI
Model	GPT-4.1 (Azure deployment)
API Version	2025-01-01-preview
Fallback Support	LM Studio / OpenAI-compatible endpoints

API Endpoints Summary

Endpoint	Method	Purpose	Question Type
/api/analyze	POST	Main intelligent routing endpoint	Auto-detect
/api/analyze/what	POST	Direct WHAT questions	WHAT
/api/analyze/why	POST	Direct WHY questions	WHY
/api/query	POST	Direct SQL query	WHAT
/api/sql-only	POST	SQL without visualization	WHAT
/api/chat	POST	General chat	N/A
/api/datasets	GET	List available datasets	N/A
/api/datasets/current	GET	Get current active dataset	N/A
/api/datasets/switch	POST	Switch to different dataset	N/A
/health	GET	Health check + agent status	N/A

Statistical Tests Mapping

```

flowchart TD
    subgraph Input["Variable Types"]
        V1[Variable 1]
        V2[Variable 2]
    end

    subgraph Tests["Statistical Tests"]
        Chi[Chi-Square Test  
Categorical × Categorical]
        TTest[Independent t-Test  
Categorical (2 groups) × Numerical]
        ANOVA[One-way ANOVA  
Categorical (3+ groups) × Numerical]
        Pearson[Pearson Correlation  
Numerical × Numerical]
        Spearman[Spearman Correlation  
Numerical × Numerical]
    end

    V1 -->|Categorical| Chi
    V2 -->|Categorical| Chi

    V1 -->|Categorical 2 groups| TTest
    V2 -->|Numerical| TTest

    V1 -->|Categorical 3+ groups| ANOVA
    V2 -->|Numerical| ANOVA

    V1 -->|Numerical| Pearson
    V2 -->|Numerical| Pearson

    V1 -->|Numerical| Spearman
    V2 -->|Numerical| Spearman

```

Example Queries

HR Employee Attrition Dataset

WHAT Questions (Descriptive)

- "What is the overall attrition rate?"
- "How many employees are in each department?"
- "What is the average monthly income by job level?"
- "Show me the distribution of years at company"
- "What percentage of employees work overtime?"
- "Compare average salaries by gender"
- "What is the job satisfaction distribution?"
- "How many employees have been promoted in the last year?"

WHY Questions (Causal)

- "Why do employees leave the company?"
- "What causes high attrition in the Sales department?"
- "Does overtime affect employee attrition?"
- "Why is job satisfaction important for retention?"
- "What factors contribute to low work-life balance?"
- "Does business travel frequency impact attrition?"

Zalando Sales Dataset

WHAT Questions (Descriptive)

- "What is the total revenue?"
- "Show revenue by product category"
- "What is the average order value?"
- "How many orders have been returned?"
- "What is the profit margin by brand?"
- "Which city generates the most revenue?"
- "Show the distribution of customer segments"
- "What is the average delivery time by city?"

WHY Questions (Causal)

- "Why do customers return products?"
- "What factors affect order profitability?"
- "Why is revenue higher in certain cities?"
- "Does discount percentage impact customer loyalty?"
- "What causes high delivery times?"
- "Why do certain brands have higher return rates?"

Cross-Dataset Insights

The system supports seamless switching between datasets, enabling:

- **Domain Comparison:** HR vs Sales analytics patterns
- **Method Validation:** Same statistical methods across different domains
- **Transferable Insights:** Apply learned patterns to new domains

Technical Implementation Details

Agent Communication Pattern

```
graph LR
    subgraph "Synchronous Flow"
        A[User Query] --> B[Planner Agent]
        B --> C[Question Type?]
        C -->|WHAT| D[Text-to-SQL Agent]
```

```
C -->|WHY| E[Hypothesis Agent]
D --> F[Visualization Agent]
E --> G[Text-to-SQL Loop]
G --> H[Visualization Loop]
H --> I[Stats Agent]
I --> J[LLM Interpretation]
end
```

Prompt Engineering Strategy

1. Few-Shot Learning

All agents use carefully crafted examples:

- **Text-to-SQL:** 5 domain-agnostic SQL patterns
- **Visualization:** 4 complete Plotly code templates
- **Hypothesis:** 3 diverse hypothesis structures
- **Stats Interpretation:** Example business insights

2. Chain-of-Thought Reasoning

Agents encouraged to "think out loud":

```
{
  "thinking_out_loud": "Step-by-step reasoning...",
  "sql_query": "SELECT ...",
  "confidence": "high"
}
```

3. Error Recovery

- **Retry Logic:** Up to 2 attempts with error feedback
- **Context Enhancement:** Previous error messages inform retry
- **Fallback Strategies:** Simplified queries when complex ones fail

Database Architecture

Multi-Schema Design

```
postgresql://localhost:5432/analytics_db
├── hr_data (schema)
│   ├── employee_attrition (table)
│   │   └── 1,470 rows x 31 columns
└── sales_data (schema)
```

```
└─ zalando_sales (table)
   └─ 10,000+ rows x 23 columns
```

Benefits:

- Logical separation of datasets
- Easy addition of new domains
- No naming conflicts
- Simplified access control

Dynamic Schema Loading

```
def get_structured_schema(db, schema_name, table_name):
    """
    Dynamically loads schema with:
    - Column names and types
    - Categorical value examples
    - Statistical summaries
    - Domain knowledge from documentation
    """
```

LLM Integration Architecture

Current: Azure OpenAI

```
AsyncAzureOpenAI(
    azure_endpoint="https://assistant-genai.openai.azure.com/",
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    api_version="2025-01-01-preview",
    deployment="gpt-4.1"
)
```

Fallback: OpenAI-Compatible APIs

- LM Studio (local models)
- Ollama
- Any OpenAI-compatible endpoint

Performance Optimizations

1. **Prompt Caching:** Reuse schema descriptions across queries
2. **Parallel Visualization:** Generate charts during data processing
3. **Lazy Loading:** Load dataset context only when needed
4. **Connection Pooling:** PostgreSQL connection management
5. **Response Streaming:** Real-time updates to frontend (planned)

Error Handling Strategy

```
try:
    # Attempt operation
    result = await agent_function()
except SpecificError as e:
    # Try alternative approach
    result = await fallback_function()
except Exception as e:
    # Log and return user-friendly error
    return {"success": False, "error": sanitize_error(e)}
```

Error Categories:

- SQL Errors → Retry with feedback
- LLM Errors → Use fallback model
- Visualization Errors → Return data only
- Statistical Errors → Skip that hypothesis

Security Considerations

1. **SQL Injection Prevention:** Parameterized queries only
2. **Input Validation:** Pydantic models for all API requests
3. **Rate Limiting:** Configurable per endpoint (planned)
4. **CORS Configuration:** Whitelist trusted origins
5. **API Key Security:** Environment variables only

File Structure Summary

```
The-Multi-Agent-Assistant-for-Smarter-Analytics/
├── README.md
├── LICENSE
├── ARCHITECTURE.md           # This file
├── backend-repo/
│   ├── requirements.txt
│   ├── .env                 # Environment variables
│   ├── create_schema_and_upload.py
│   ├── list_databases.py
│   ├── load_to_postgres.py
│   └── app/
│       ├── __init__.py
│       ├── main.py          # FastAPI entry point
│       ├── config.py        # Settings
│       └── api/
│           └── routes.py    # API endpoints
│       ├── services/        # Agent implementations
│       └── prompts/         # LLM prompt templates
```



System Capabilities

Question Classification & Routing

The **Planner Agent** intelligently classifies questions into two categories:

Question Type	Analytics Type	Example	Agent Pipeline
WHAT	Descriptive	"What is the attrition rate?"	Text-to-SQL → Visualization
WHY	Causal	"Why do employees leave?"	Hypothesis → Text-to-SQL → Viz → Stats

Supported Analysis Types

1. Descriptive Analytics (WHAT Questions)

- **Basic Metrics:** Counts, averages, sums, percentages
- **Distributions:** Category breakdowns, frequency analysis
- **Comparisons:** Group-by analysis, cross-tabulations
- **Aggregations:** Multi-level summaries, pivot operations
- **Time Series:** Temporal trends and patterns

Example Queries:

- "What is the total revenue by category?"
- "How many employees work overtime?"
- "Show the distribution of salaries by department"
- "What is the average order value by customer segment?"

2. Causal Analytics (WHY Questions)

- **Hypothesis Generation:** Automated creation of testable hypotheses
- **Bivariate Analysis:** Two-variable relationship testing
- **Statistical Testing:** Chi-square, t-test, ANOVA, correlation
- **Effect Size Calculation:** Practical significance assessment
- **Business Interpretation:** AI-generated insights and recommendations

Example Queries:

- "Why do employees leave the company?"
- "What factors affect customer satisfaction?"
- "Why is revenue declining in certain regions?"
- "What causes high return rates?"

Statistical Test Selection

The system automatically selects the appropriate statistical test based on variable types:

Variable 1 Type	Variable 2 Type	Statistical Test	Use Case
Categorical	Categorical	Chi-Square Test	Association between categories
Categorical (2 groups)	Numerical	Independent t-Test	Mean difference between 2 groups
Categorical (3+ groups)	Numerical	One-Way ANOVA	Mean differences across groups
Numerical	Numerical	Pearson Correlation	Linear relationship strength
Numerical (non-linear)	Numerical	Spearman Correlation	Monotonic relationship

Visualization Intelligence

The **Visualization Agent** automatically selects chart types based on data characteristics:

Data Shape	Chart Type	When Used
1 row, 1 value	Indicator Card	Single KPI/metric
N rows, 1 categorical, 1 numerical	Bar Chart	Category comparisons
N rows, 2+ categories	Grouped Bar Chart	Multi-category analysis
Percentage/proportion data	Pie Chart	Part-to-whole relationships
2 numerical variables	Scatter Plot	Correlation analysis
Ordinal scale (ratings)	Bar Chart	Rating distributions

Data Shape	Chart Type	When Used
Time series data	Line Chart	Temporal trends

Automatic Formatting

Smart number formatting based on column name patterns:

- **Counts** (`_count`, `total_`, `num_`): 1,470
- **Percentages** (`_percent`, `_rate`): 16.12%
- **Averages** (`avg_`, `mean_`): 6,502.93
- **Currency** (`_income`, `_salary`, `_revenue`, `_cost`): \$6,502.93

Recent Enhancements (December 2025 - January 2026)

1. Multi-Dataset Support

- **Dataset Manager Service:** Centralized management of multiple datasets
- **Dynamic Schema Switching:** Seamlessly switch between HR and Sales data
- **Dataset-Specific Context:** Automatic loading of data dictionaries and KPI documentation
- **Frontend Integration:** UI controls for dataset selection with live switching

2. Enhanced Agent Activity Monitor

- **Structured Logging:** New `logger.py` utility with timing and performance tracking
- **Error Tracking:** Detailed error logs with stack traces and context
- **Performance Metrics:** Query execution time, agent duration tracking
- **Visual Improvements:** Color-coded log levels, expandable error details
- **Agent Grouping:** Timeline and grouped views of agent activities

3. Improved Prompt Engineering

- **Text-to-SQL Enhancements:**
 - Query type pattern mapping (COUNT, AVG, SUM, etc.)
 - PostgreSQL type casting rules for ROUND() errors
 - Column alias naming conventions
 - Categorical value examples in schema
- **Visualization Intelligence:**
 - Data shape → chart type mapping
 - Ordinal scale detection for rating columns
 - Automatic number formatting based on column names
 - Complete Plotly code templates
- **Hypothesis Generation:**
 - Outcome variable detection
 - Diversity requirements across factor categories

- Domain-agnostic hypothesis patterns

- **Statistical Interpretation:**

- LLM-powered business insight generation
- Plain English explanations of statistical results
- Actionable recommendations from test results

4. Azure OpenAI Integration

- Migrated from LM Studio to Azure OpenAI
- Using GPT-4.1 deployment
- Enhanced reliability and performance
- Backward compatible with OpenAI-compatible endpoints

5. Statistical Analysis Improvements

- **LLM Interpretation Layer:** Converts raw statistics to business insights
- **Enhanced Test Results:** Includes effect sizes, confidence intervals
- **User-Friendly Output:** Plain English explanations with recommendations
- **Comprehensive Testing:** Chi-square, t-test, ANOVA, Pearson, Spearman

6. User Experience Enhancements

- **Dataset Banners:** Visual confirmation of dataset switches
- **Example Prompts:** Dataset-specific example questions
- **Responsive Design:** Improved mobile and desktop layouts
- **Real-time Feedback:** Live agent status updates
- **Error Handling:** Graceful degradation with helpful error messages

System Metrics

Code Statistics

- **Total Lines of Code:** ~15,000+
- **Backend (Python):** ~8,000 lines
- **Frontend (React/TypeScript):** ~7,000 lines
- **Prompt Templates:** ~2,500 lines
- **Test Coverage:** 100+ test questions per dataset

Performance Benchmarks

- **WHAT Question Response Time:** 3-8 seconds
- **WHY Question Response Time:** 15-30 seconds (3 hypotheses)
- **SQL Generation Success Rate:** >95%
- **Visualization Generation Rate:** >90%
- **Statistical Test Accuracy:** 100% (validated against SciPy)

Supported Datasets

- 1. **HR Employee Attrition:** 1,470 records, 31 columns
- 2. **Zalando Sales:** E-commerce transactions, 25+ columns
- 3. **Extensible:** Easy addition of new datasets via Dataset Manager

Extensibility & Roadmap

Adding New Datasets

The system is designed for easy dataset extension:

```
# In dataset_manager.py
self.datasets["new_dataset"] = DatasetInfo(
    id="new_dataset",
    name="New Dataset Name",
    description="Dataset description",
    schema_name="new_schema",
    main_table="main_table_name",
    data_dictionary_path="path/to/dictionary.csv",
    kpi_documentation_path="path/to/kpi_docs.txt"
)
```

Required Files:

- 1. **Data Dictionary CSV:** Column definitions and types
- 2. **KPI Documentation TXT:** Domain knowledge and business context
- 3. **SQL Schema:** PostgreSQL table structure

Future Enhancements

Phase 1: Advanced Analytics (Q1 2026)

- ☐ Time series forecasting with Prophet/ARIMA
- ☐ Clustering analysis for segmentation
- ☐ Anomaly detection in metrics
- ☐ Multi-variate regression analysis

Phase 2: Performance & Scalability (Q2 2026)

- ☐ Response streaming for real-time updates
- ☐ Redis caching for frequently accessed queries
- ☐ Asynchronous hypothesis testing
- ☐ Batch query processing

Phase 3: Enhanced Intelligence (Q2-Q3 2026)

- ☐ Multi-step reasoning for complex questions
- ☐ Automatic insight generation (proactive analytics)
- ☐ Natural language report generation

- ☐ Cross-dataset comparative analysis

Phase 4: Enterprise Features (Q3-Q4 2026)

- ☐ User authentication and authorization
- ☐ Query history and saved analysis
- ☐ Scheduled reports and alerts
- ☐ Export to PDF/PowerPoint
- ☐ Collaborative annotations
- ☐ Role-based access control

Customization Points

The architecture supports customization at multiple levels:

1. **Prompt Templates:** Modify in `prompts/prompts.py`
2. **Statistical Tests:** Add new tests in `utils/stats_utils.py`
3. **Visualization Types:** Extend chart logic in `services/visualization_agent.py`
4. **LLM Models:** Switch models via environment variables
5. **Database:** Support for other SQL databases (MySQL, SQLite)

Integration Options

The system can be integrated into:

- **Business Intelligence Platforms:** Embed as analytics module
- **Data Warehouses:** Direct connection to enterprise data
- **Slack/Teams:** Chatbot interface for analytics
- **Jupyter Notebooks:** Python SDK for data scientists
- **REST API:** Standalone microservice

Deployment Guide

Local Development

```
# Backend
cd backend-repo
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Frontend
cd frontend-repo
npm install
npm run dev
```

Production Deployment

Option 1: Docker Compose

```
version: '3.8'
services:
  backend:
    build: ./backend-repo
    ports:
      - "8000:8000"
    environment:
      - AZURE_OPENAI_API_KEY=${API_KEY}
      - DATABASE_URL=${DB_URL}

  frontend:
    build: ./frontend-repo
    ports:
      - "80:80"
    depends_on:
      - backend

  postgres:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data
```

Option 2: Cloud Platform

- **Azure:** App Service + PostgreSQL
- **AWS:** ECS + RDS
- **GCP:** Cloud Run + Cloud SQL
- **Heroku:** Web + Postgres add-on

Environment Variables

```
# Required
AZURE_OPENAI_API_KEY=your_key_here
AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com/
AZURE_OPENAI_DEPLOYMENT=gpt-4.1

# Optional
DATABASE_URL=postgresql://user:pass@host:5432/db
DEFAULT_DATASET=hr_data
LOG_LEVEL=INFO
```